

Триггеры (PostgreSQL)

1. Проверка даты выселения (должна быть NULL или больше даты заезда):

```
CREATE OR REPLACE FUNCTION trg_func_check_dates()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.check_out_date IS NOT NULL AND NEW.check_out_date <=
NEW.check_in_date THEN
        RAISE EXCEPTION 'Ошибка: Дата выезда должна быть больше
даты заезда.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_dates
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW EXECUTE FUNCTION trg_func_check_dates();
```

```
-- Дата выезда раньше даты заезда:
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (5, 5, '2026-05-10', '2026-05-09', 1000);
```

```
-- Дата выезда совпадает с датой заезда (0 ночей):
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (5, 5, '2026-05-10', '2026-05-10', 1000);
```

```
-- Правильные даты:  
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost)  
VALUES (5, 5, '2026-05-10', '2026-05-15', 30000);
```

Результат:

```
[P0001] ОШИБКА: Ошибка: Дата выезда должна быть больше даты заезда.  
Где: функция PL/pgSQL trg_func_check_dates(), строка 4, оператор RAISE
```

2. Автоподстановка стоимости услуги, если она не указана или равна нулю:

```
CREATE OR REPLACE FUNCTION trg_func_set_service_price()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.fixed_price IS NULL OR NEW.fixed_price = 0 THEN  
        SELECT base_price INTO NEW.fixed_price  
        FROM services  
        WHERE service_id = NEW.service_id;  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_set_service_price  
BEFORE INSERT OR UPDATE ON booking_services  
FOR EACH ROW EXECUTE FUNCTION trg_func_set_service_price();
```

-- Передаем цену 0. Заменит 0 на 500 (цена завтрака).

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)
```

```
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 1, 2, 0);
```

-- Передаем цену NULL. Заменит NULL на 300 (цена стирки).

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)
```

```
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 3, 1, NULL);
```

-- Передаем свою цену (например, скидка). Оставит 150.

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)
```

```
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 1, 1, 150);
```

-- Результат:

```
SELECT bs.*
```

```
FROM booking_services bs
```

```
JOIN bookings b ON bs.booking_id = b.booking_id
```

```
WHERE b.client_id = 5;
```

Результат:

	record_id	booking_id	service_id	quantity	fixed_price
1	9	6	1	2	500.00
2	10	6	3	1	300.00
3	11	6	1	1	150.00

3. Проверка, что номер не занят никем другим:

```
CREATE OR REPLACE FUNCTION trg_func_prevent_double_booking()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF EXISTS (
```

```
        SELECT 1 FROM bookings
```

```
        WHERE room_id = NEW.room_id
```

```
        AND booking_id IS DISTINCT FROM NEW.booking_id
```

```
        AND check_in_date < NEW.check_out_date
```

```
        AND check_out_date > NEW.check_in_date
```

```
    ) THEN
```

```
        RAISE EXCEPTION 'Ошибка: В этом номере уже проживают  
гости на выбранные даты!';
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_prevent_double_booking
```

```
BEFORE INSERT OR UPDATE ON bookings
```

```
FOR EACH ROW EXECUTE FUNCTION
```

```
trg_func_prevent_double_booking();
```

```
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost) VALUES  
(3, 1, '2026-03-03', '2026-03-05', 3000.00);
```

Результат:

```
[P0001] ОШИБКА: Ошибка заселения: Превышена вместимость номера! Всего мест: 1, Занято: 1  
Где: функция PL/pgSQL trg_func_check_capacity(), строка 18, оператор RAISE
```

4. Проверка поля “Пол” (только “М” или “Ж”, в нижнем регистре):

```
ALTER TABLE clients ADD COLUMN IF NOT EXISTS gender  
VARCHAR(1);
```

```
CREATE OR REPLACE FUNCTION trg_func_check_gender()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.gender IS NOT NULL AND NEW.gender NOT IN ('м', 'ж')  
THEN  
        RAISE EXCEPTION 'Ошибка: Поле "Пол" может содержать  
только значения "м" или "ж" в нижнем регистре.';  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_gender  
BEFORE INSERT OR UPDATE ON clients  
FOR EACH ROW EXECUTE FUNCTION trg_func_check_gender();
```

-- Заглавная буква 'M'.

```
UPDATE clients SET gender = 'M' WHERE client_id = 5;
```

-- Недопустимый символ (например, 'x').

```
UPDATE clients SET gender = 'x' WHERE client_id = 5;
```

-- Строчная 'm'.

```
UPDATE clients SET gender = 'm' WHERE client_id = 5;
```

Результат:

```
[P0001] ОШИБКА: Ошибка: Поле "Пол" может содержать только значения "м" или "ж" в нижнем регистре.  
Где: функция PL/pgSQL trg_func_check_gender(), строка 4, оператор RAISE
```

5. Предотвращение заселения людей свыше вместимости номера:

```
DROP TRIGGER IF EXISTS trg_prevent_double_booking ON bookings;
```

```
CREATE OR REPLACE FUNCTION trg_func_check_capacity()
```

```
RETURNS TRIGGER AS $$
```

```
DECLARE
```

```
    v_room_capacity INT;
```

```
    v_overlapping_guests INT;
```

```
BEGIN
```

```
    SELECT capacity INTO v_room_capacity
```

```
    FROM rooms
```

```
    WHERE room_id = NEW.room_id;
```

```
    SELECT COUNT(*) INTO v_overlapping_guests
```

```
    FROM bookings
```

```

WHERE room_id = NEW.room_id
AND booking_id IS DISTINCT FROM NEW.booking_id
AND check_in_date < NEW.check_out_date
AND check_out_date > NEW.check_in_date;

IF (v_overlapping_guests + 1) > v_room_capacity THEN
    RAISE EXCEPTION 'Ошибка заселения: Превышена вместимость
номера! Всего мест: %, Занято: %',
        v_room_capacity, v_overlapping_guests;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_capacity
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW EXECUTE FUNCTION trg_func_check_capacity();

-- Селим 2-го человека на те же даты (осталось 2 места).
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (1, 5, '2026-05-10', '2026-05-15', 5000);

-- Селим 3-го человека (осталось 1 место).
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (2, 5, '2026-05-10', '2026-05-15', 5000);

```

```
-- Селим 4-го человека (мест больше нет, номер заполнен).  
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost)  
VALUES (3, 5, '2026-05-10', '2026-05-15', 5000);
```

```
-- Пытаемся поселить 5-го человека в 4-местный номер.  
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost)  
VALUES (4, 5, '2026-05-10', '2026-05-15', 5000);
```

Результат:

```
[P0001] ОШИБКА: Ошибка: В этом номере уже проживают гости на выбранные даты!  
Где: функция PL/pgSQL trg_func_prevent_double_booking(), строка 10, оператор RAISE
```

Триггеры (MySQL)

1. Проверка даты выселения (должна быть NULL или больше даты заезда):

```
DELIMITER //
```



```
CREATE TRIGGER trg_check_dates_ins  
BEFORE INSERT ON bookings  
FOR EACH ROW  
BEGIN  
    IF NEW.check_out_date IS NOT NULL AND NEW.check_out_date <=  
NEW.check_in_date THEN  
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка:  
Дата выезда должна быть больше даты заезда.';  
    END IF;
```


END//

```
CREATE TRIGGER trg_check_dates_upd
BEFORE UPDATE ON bookings
FOR EACH ROW
BEGIN
    IF NEW.check_out_date IS NOT NULL AND NEW.check_out_date <=
NEW.check_in_date THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка:
Дата выезда должна быть больше даты заезда.';
    END IF;
END//
```

DELIMITER ;

```
-- Дата выезда раньше даты заезда:
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost) VALUES (5, 5, '2026-05-10', '2026-05-09', 1000);

-- Дата выезда совпадает с датой заезда (0 ночей):
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost) VALUES (5, 5, '2026-05-10', '2026-05-10', 1000);

-- Правильные даты:
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (5, 5, '2026-05-10', '2026-05-15', 30000);
```

Результат:

[45000][1644] Ошибка: Дата выезда должна быть больше даты заезда.

2. Автоподстановка стоимости услуги, если она не указана или равна нулю:

```
DELIMITER //
```

```
CREATE TRIGGER trg_set_service_price_ins
BEFORE INSERT ON booking_services
FOR EACH ROW
BEGIN
    IF NEW.fixed_price IS NULL OR NEW.fixed_price = 0 THEN
        SET NEW.fixed_price = (SELECT base_price FROM services
WHERE service_id = NEW.service_id);
    END IF;
END//
```

```
CREATE TRIGGER trg_set_service_price_upd
BEFORE UPDATE ON booking_services
FOR EACH ROW
BEGIN
    IF NEW.fixed_price IS NULL OR NEW.fixed_price = 0 THEN
        SET NEW.fixed_price = (SELECT base_price FROM services
WHERE service_id = NEW.service_id);
    END IF;
END//
```

```
DELIMITER ;
```

-- Передаем цену 0. Заменит 0 на 500 (цена завтрака).

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)  
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 1, 2, 0);
```

-- Передаем цену NULL. Заменит NULL на 300 (цена стирки).

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)  
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 3, 1, NULL);
```

-- Передаем свою цену (например, скидка). Оставит 150.

```
INSERT INTO booking_services (booking_id, service_id, quantity,  
fixed_price)  
VALUES ((SELECT MAX(booking_id) FROM bookings WHERE  
client_id = 5), 1, 1, 150);
```

-- Результат:

```
SELECT bs.*  
FROM booking_services bs  
JOIN bookings b ON bs.booking_id = b.booking_id  
WHERE b.client_id = 5;
```

Результат:

	record_id	booking_id	service_id	quantity	fixed_price
1	4	6	1	2	500.00
2	5	6	3	1	300.00
3	6	6	1	1	150.00

3. Проверка, что номер не занят никем другим:

DELIMITER //

```

CREATE TRIGGER trg_prevent_double_booking_ins
BEFORE INSERT ON bookings
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM bookings
        WHERE room_id = NEW.room_id
        AND (NEW.booking_id IS NULL OR booking_id !=
NEW.booking_id)
        AND check_in_date < NEW.check_out_date
        AND check_out_date > NEW.check_in_date
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка: В
этом номере уже проживают гости на выбранные даты!';
    END IF;
END//

```

```

CREATE TRIGGER trg_prevent_double_booking_upd
BEFORE UPDATE ON bookings
FOR EACH ROW
BEGIN

```

```
IF EXISTS (  
    SELECT 1 FROM bookings  
    WHERE room_id = NEW.room_id  
        AND (NEW.booking_id IS NULL OR booking_id !=  
NEW.booking_id)  
        AND check_in_date < NEW.check_out_date  
        AND check_out_date > NEW.check_in_date  
    ) THEN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка: В  
этом номере уже проживают гости на выбранные даты!';  
END IF;  
END//
```

DELIMITER ;

```
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost) VALUES (3, 1, '2026-03-03', '2026-03-05',  
3000.00);
```

Результат:

[45000][1644] Ошибка: В этом номере уже проживают гости на выбранные даты!

4. Проверка поля “Пол” (только “М” или “Ж”, в нижнем регистре):

```
ALTER TABLE clients ADD COLUMN gender VARCHAR(1);
```

DELIMITER //

```
CREATE TRIGGER trg_check_gender_ins
BEFORE INSERT ON clients
FOR EACH ROW
BEGIN
    -- BINARY делает проверку чувствительной к регистру (М != м)
    IF NEW.gender IS NOT NULL AND BINARY NEW.gender NOT IN
('м', 'ж') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка:
Поле "Пол" может содержать только значения "м" или "ж" в нижнем
регистре.';
    END IF;
END//
```

```
CREATE TRIGGER trg_check_gender_upd
BEFORE UPDATE ON clients
FOR EACH ROW
BEGIN
    IF NEW.gender IS NOT NULL AND BINARY NEW.gender NOT IN
('м', 'ж') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Ошибка:
Поле "Пол" может содержать только значения "м" или "ж" в нижнем
регистре.';
    END IF;
END//
```

```
DELIMITER ;
```

```
-- Заглавная буква 'М'.
```

```
UPDATE clients SET gender = 'М' WHERE client_id = 5;
```

-- Недопустимый символ.

```
UPDATE clients SET gender = 'x' WHERE client_id = 5;
```

-- Строчная 'м'.

```
UPDATE clients SET gender = 'м' WHERE client_id = 5;
```

Результат:

```
[45000][1644] Ошибка: Поле "Пол" может содержать только значения "м" или "ж" в нижнем регистре.
```

5. Предотвращение заселения людей свыше вместимости номера:

```
DROP TRIGGER IF EXISTS trg_prevent_double_booking_ins;
```

```
DROP TRIGGER IF EXISTS trg_prevent_double_booking_upd;
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_check_capacity_ins
```

```
BEFORE INSERT ON bookings
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE v_room_capacity INT;
```

```
    DECLARE v_overlapping_guests INT;
```

```
    DECLARE err_msg VARCHAR(255);
```

```
    SELECT capacity INTO v_room_capacity FROM rooms WHERE  
room_id = NEW.room_id;
```

```

SELECT COUNT(*) INTO v_overlapping_guests
FROM bookings
WHERE room_id = NEW.room_id
      AND (NEW.booking_id IS NULL OR booking_id != NEW.booking_id)
      AND check_in_date < NEW.check_out_date
      AND check_out_date > NEW.check_in_date;

IF (v_overlapping_guests + 1) > v_room_capacity THEN
    -- Формируем текст ошибки через CONCAT
    SET err_msg = CONCAT('Ошибка заселения: Превышена
вместимость номера! Всего мест: ', v_room_capacity, ', Занято: ',
v_overlapping_guests);
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = err_msg;
END IF;
END//

```

```

CREATE TRIGGER trg_check_capacity_upd
BEFORE UPDATE ON bookings
FOR EACH ROW
BEGIN
    DECLARE v_room_capacity INT;
    DECLARE v_overlapping_guests INT;
    DECLARE err_msg VARCHAR(255);

    SELECT capacity INTO v_room_capacity FROM rooms WHERE
room_id = NEW.room_id;

    SELECT COUNT(*) INTO v_overlapping_guests
FROM bookings

```



```

WHERE room_id = NEW.room_id
      AND (NEW.booking_id IS NULL OR booking_id != NEW.booking_id)
      AND check_in_date < NEW.check_out_date
      AND check_out_date > NEW.check_in_date;

IF (v_overlapping_guests + 1) > v_room_capacity THEN
    SET err_msg = CONCAT('Ошибка заселения: Превышена
вместимость номера! Всего мест: ', v_room_capacity, ', Занято: ',
v_overlapping_guests);
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = err_msg;
END IF;
END//

DELIMITER ;

-- Селим 2-го человека на те же даты (осталось 2 места).
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (1, 5, '2026-05-10', '2026-05-15', 5000);

-- Селим 3-го человека (осталось 1 место).
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (2, 5, '2026-05-10', '2026-05-15', 5000);

-- Селим 4-го человека (мест больше нет, номер заполнен).
INSERT INTO bookings (client_id, room_id, check_in_date,
check_out_date, total_cost)
VALUES (3, 5, '2026-05-10', '2026-05-15', 5000);

```

-- Пытаемся поселить 5-го человека в 4-местный номер.

```
INSERT INTO bookings (client_id, room_id, check_in_date,  
check_out_date, total_cost) VALUES (4, 5, '2026-05-10', '2026-05-15', 5000);
```

Результат:

```
[45000][1644] Ошибка заселения: Превышена вместимость номера! Всего мест: 4, Занято: 4
```